

Software Architecture and Design Specification

Blood Bank Management System

Field	Value
Project	Blood Bank Management System
Version	1.0
Authors	Team Number 2
Date	03-10-2026
Status	Draft

Revision History

Version	Date	Author	Change Summary
1.0	03-10-2026	Student Team	Initial Software Architecture and Design Specification.

Approvals

Role	Name	Signature / Date
Course Coordinator / Instructor	Sheela Devi	
Team Representative	Khush Bhardwaj	Khush Bhardwaj

1. Introduction

1.1 Purpose

This document specifies the architecture and design of the Blood Bank Management System. It translates the submitted SRS into a practical Django-based architecture and defines components, interactions, security, interfaces, and design decisions.

1.2 Scope

The architecture supports registration/authentication, role-based access for Administrator, Donor, and Patient, profiles, donation and blood-request workflows, approval/rejection, inventory updates, history, dashboard information, and basic expiry tracking. Physical collection, laboratory testing, transportation, and clinical decision-making are outside the system boundary.

1.3 Audience

Student developers, testers, course instructors, maintainers, and project reviewers.

1.4 Definitions

Term	Definition
Django	Python web framework used for the backend.

MVT	Django Model-View-Template architectural pattern.
ORM	Django Object-Relational Mapper.
RBAC	Role-Based Access Control.
Stock	Recorded available blood units by blood group.
SRS	Software Requirements Specification.
STP	Software Test Plan.

2. Document Overview

2.1 How to Use This Document

Sections 3 and 4 describe the architecture and detailed design. The diagrams should be read with the submitted SRS and Test Plan. Component names follow the modular structure of the selected Django reference project.

2.2 Related Documents

- Blood Bank Management System - Software Requirements Specification (SRS), Version 1.0.
- Blood Bank Management System - Software Test Plan (STP), Version 1.0.
- Software Engineering Mini-Project Deliverables Part-1.
- Reference GitHub repository: [pratyushsrivastava500/Blood-Bank-Management-System](https://github.com/pratyushsrivastava500/Blood-Bank-Management-System).

3. Architecture

3.1 Goals & Constraints

- Keep the implementation simple enough for a student mini-project.
- Separate presentation, application, model/data, and persistence concerns.
- Reuse Django authentication and ORM facilities.
- Support Administrator, Donor, and Patient roles.
- Remain consistent with the SRS and reference repository structure.
- Use SQLite for initial development; migration to another relational database is possible if required.

3.2 Stakeholders & Concerns

Stakeholder	Main Concern
Administrator	Approvals, stock management, user management, dashboard.
Donor	Registration, donation submission, status, history.
Patient	Registration, blood requests, status, history.
Developer / Maintainer	Modularity and maintainability.
Tester / Instructor	Traceability, security, predictable workflows.

3.3 Component (UML) Diagram

The architecture follows the reference repository's documented four-layer structure: Django web handling and templates above the application modules, model/ORM logic below them, and SQLite persistence at the bottom.

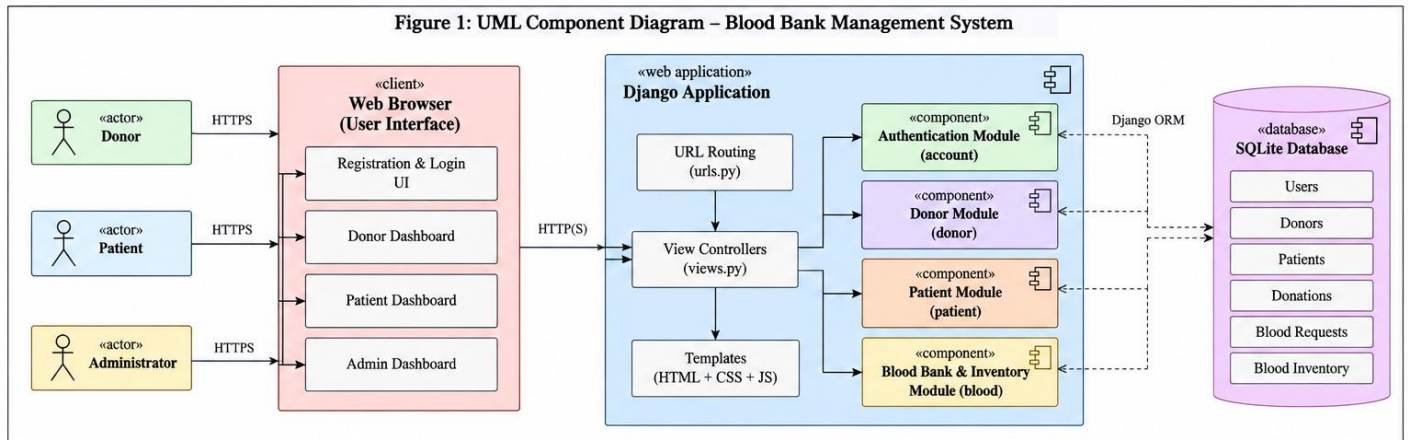


Figure 1. Layered component architecture of the Blood Bank Management System.

3.4 Component Descriptions

Component	Responsibility
Django URL Routing	Maps browser requests to views.
Authentication / Authorization	Handles login sessions, passwords, and role-restricted access.
blood app	Admin blood stock, request/donation approval, dashboard, and related models/views.
donor app	Donor registration, profile, donation submission, history, and donor requests.
patient app	Patient registration, profile, requests, and request tracking.
Models / ORM	Represents Stock, BloodRequest, BloodDonate, Donor, Patient, and User.
Templates / Static	Browser UI using HTML/CSS/Bootstrap and JavaScript.
SQLite Database	Persists application, authentication, inventory, donation, and request records.

3.5 Chosen Architecture Pattern and Rationale

A layered architecture implemented through Django's Model-View-Template approach is selected. The presentation layer contains templates/static assets; the application layer contains routing, views, forms, and access control; the model/data layer contains domain models and ORM operations; and SQLite provides persistence. A microservices architecture is intentionally not used because it would add unnecessary deployment and operational complexity for this project.

3.6 Technology Stack & Data Stores

Area	Technology / Choice
Backend	Python 3.x with Django
Frontend	Django Templates, HTML5, CSS3, Bootstrap, JavaScript
Authentication	Django Authentication System

Data Access	Django ORM
Database	SQLite3 for development/reference implementation
Static Assets	Django Static Files
Deployment	Django development server for project testing

3.7 Risks & Mitigations

Risk	Mitigation
Role permissions overlap	Test Admin/Donor/Patient access separately and keep role checks consistent.
Stock inconsistency	Update stock only through approval workflows and verify with data-integrity tests.
Reference-code changes become hard to manage	Keep modules separated and make small changes.
Migration/environment issues	Use Django migrations and a reproducible requirements setup.
Sensitive data exposure	Use Django authentication, CSRF, input validation, and HTTPS for network deployment.

3.8 Traceability to Requirements

SRS Area	Architecture Component(s)
BB-F-001 to BB-F-003	Django Authentication, routing, donor/patient/blood views
BB-F-004 to BB-F-007	donor app, blood app, BloodDonate model
BB-F-008 to BB-F-011	patient app, blood app, BloodRequest and Stock models
BB-F-012 to BB-F-014	blood app, Stock model, Django ORM
BB-F-015 to BB-F-018	blood/donor/patient apps, templates, database models
BB-NF-001 to BB-NF-006	Layered architecture, Django, ORM/database, templates, validation
BB-SR-001 to BB-SR-005	Django Auth, authorization, forms/validation, secure deployment

3.9 Security Architecture

- Authentication uses Django's authentication system and sessions.
- Passwords are handled using Django password hashing rather than plain-text storage.
- Administrator, Donor, and Patient operations are separated by role-specific views/routes.
- Django CSRF protection is used for state-changing browser forms.
- Django forms/model validation reject invalid or incomplete input.
- Only project-required donor, patient, donation, request, and inventory data is stored.
- HTTPS is required when the application is deployed over a network, as specified in the SRS.

3.10 Architecture Decision Records (ADRs)

ADR	Decision	Reason
ADR-01	Use Django MVT / layered architecture	Simple separation of concerns and directly supported by the selected stack.
ADR-02	Use SQLite for development	Low setup overhead and adequate for the

		academic scope.
ADR-03	Keep Admin, Donor, Patient as logical roles/modules	Matches the SRS without introducing a complex identity service.
ADR-04	Use Django ORM for normal database operations	Reduces database-specific code and integrates with models/migrations.

4. Design

4.1 Design Overview

Browser requests enter through Django URL routing, are handled by role-appropriate views/forms, operate on models through the ORM, and persist to SQLite. Templates render the resulting state back to the user. The two principal workflows are donation approval with stock increase and blood-request approval with stock deduction.

4.2 UML Sequence Diagrams

These diagrams cover the two main transactional flows in the SRS and match the Admin, Donor, Patient, Stock, BloodDonate, and BloodRequest concepts used by the reference implementation.

Figure 2: UML Sequence Diagram – Donation Submission and Approval

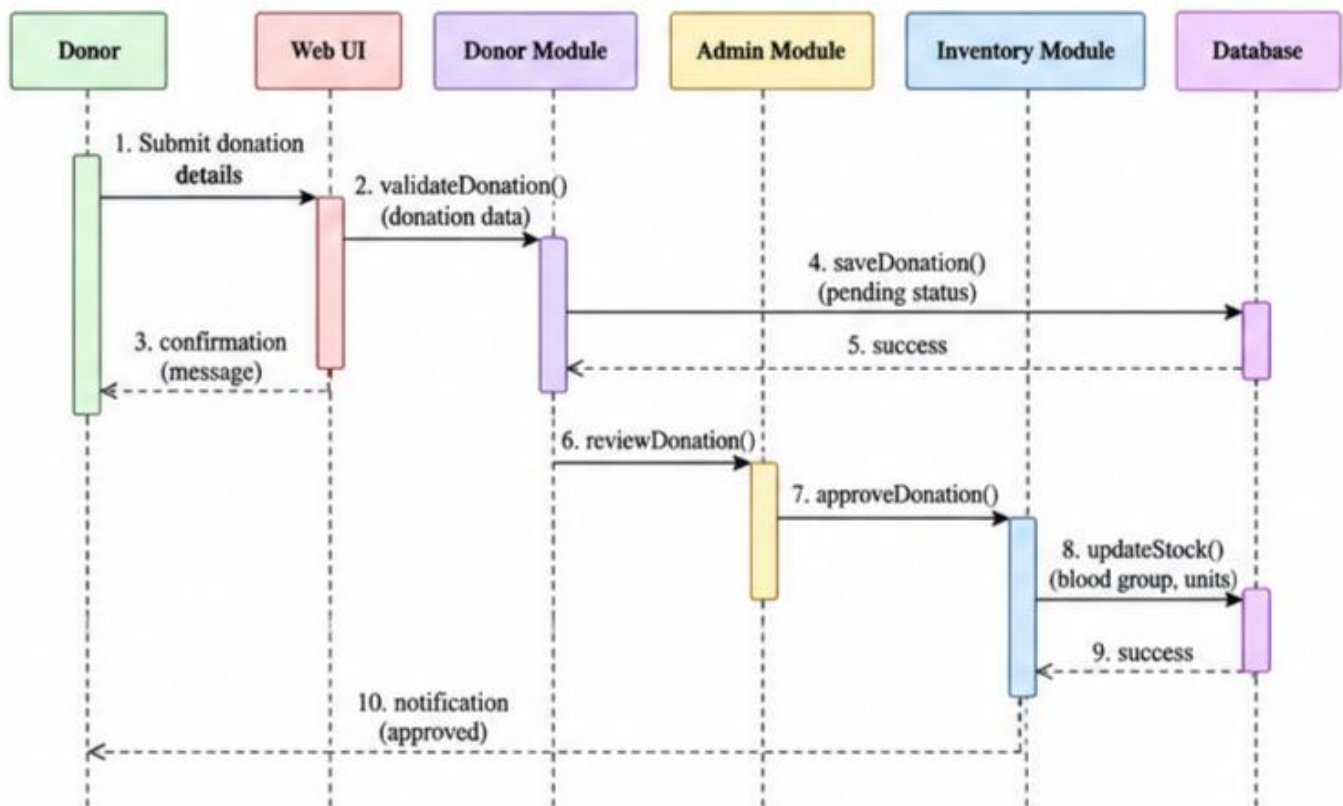


Figure 2. Donor donation submission followed by administrator approval and stock update.

Figure 3: UML Sequence Diagram – Blood Request and Approval

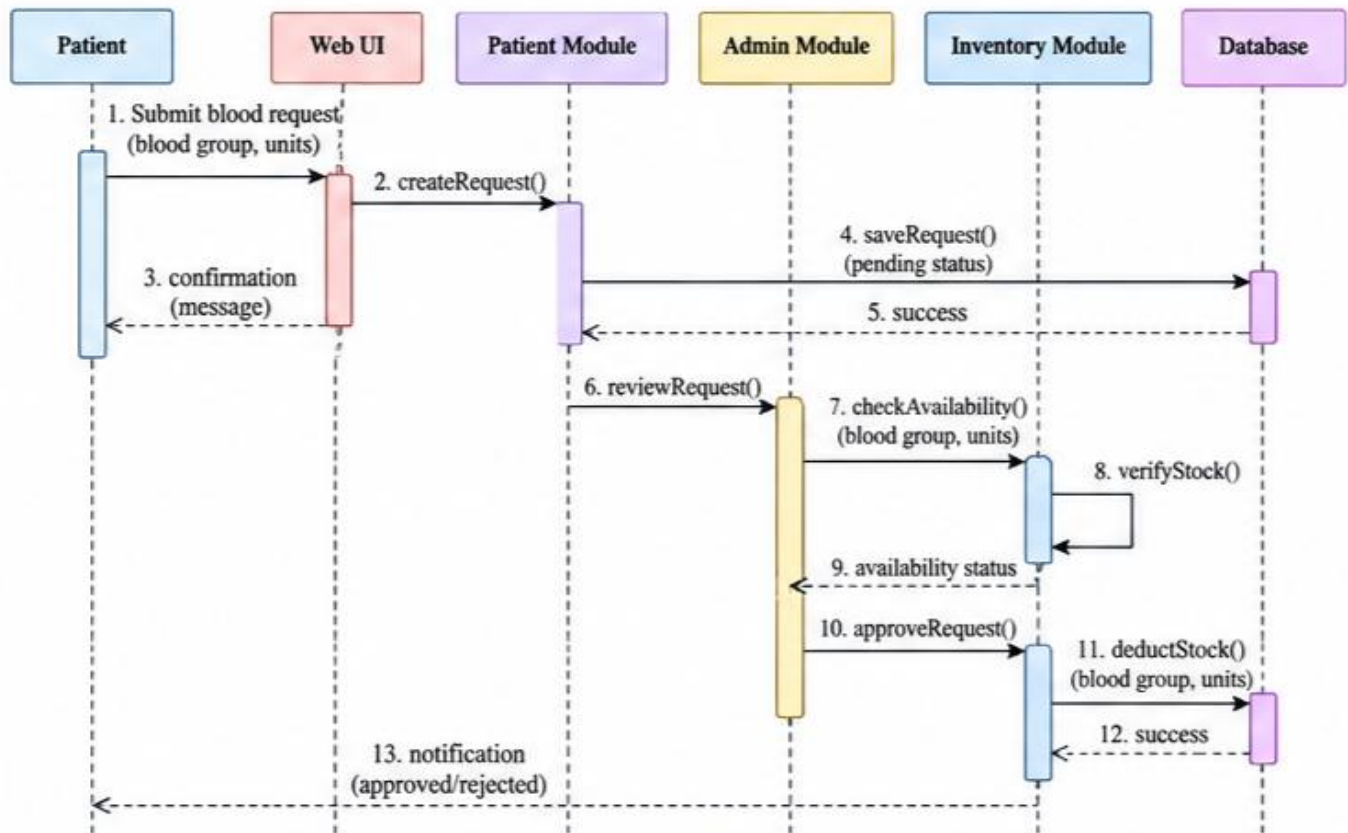


Figure 3. Patient blood request followed by administrator approval and stock deduction.

4.3 API / Interface Design

The reference implementation is primarily a Django server-rendered web application rather than a separate REST service. Therefore, the following are documented as internal web interfaces between the browser, Django views, models, and database.

Interface	Method / Input	Processing	Output
Donor Donation	Authenticated POST form	Validate and create pending BloodDonate record	Confirmation/status page
Admin Donation Approval	Authenticated Admin action	Update donation status; approved donation increases Stock	Updated status and stock
Patient Blood Request	Authenticated POST form	Validate and create BloodRequest	Confirmation/status page
Admin Request Approval	Authenticated Admin action	Check stock; approved request deducts Stock	Updated status and remaining stock

4.4 Error Handling, Logging & Monitoring

- Form validation errors return to the relevant page with readable messages.
- Unauthorized users are denied or redirected without protected data exposure.
- Invalid stock operations are rejected so inventory does not become negative.

- Database/application failures should not expose sensitive implementation details.
- Development logs may be used for diagnosis; credentials and sensitive data must not be logged.
- Basic monitoring includes application availability, authentication failures, and stock/request workflow errors.

4.5 UX Design

- Separate navigation/dashboards for Administrator, Donor, and Patient.
- Forms use clear labels, required fields, and validation feedback.
- Donation/request status is shown as Pending, Approved, or Rejected.
- Blood availability is displayed by blood group for administrative monitoring.
- Bootstrap-based responsive layouts are used where supported by the reference UI.

4.6 Data Model Overview

Entity	Key Data / Relationships
User	Django authentication account; linked to Donor or Patient profile.
Donor	User relationship, blood group, contact information, donation relationship.
Patient	User relationship, basic/medical information, request relationship.
BloodDonate	Donor, blood group, units, status, date, donation details.
BloodRequest	Requester, blood group, units, reason/details, status, date.
Stock	Blood group and current available units.

4.7 Open Issues & Next Steps

- Confirm the final reference-code version before implementation.
- Replace reference-project branding/text with project-specific content.
- Implement only the SRS-approved scope; avoid optional features that increase testing effort.
- Complete detailed test execution and update the RTM after implementation.
- Configure HTTPS if the final system is deployed beyond local development.
- Review the architecture after modifications so diagrams remain consistent with the actual code.

5. Appendices

5.1 Glossary

Term	Meaning
Admin	Administrator managing approvals, users, and stock.
Donor	Registered user submitting donation information.
Patient	Registered user submitting blood requests.
BloodDonate	Record representing a donor's submitted donation.
BloodRequest	Record representing a request for blood.
Stock	Current blood availability by blood group.
MVT	Django Model-View-Template pattern.

5.2 References

- Blood Bank Management System - SRS, Version 1.0.
- Blood Bank Management System - STP, Version 1.0.
- Software Engineering Mini-Project Deliverables Part-1.
- Reference repository: pratyushsrivastava500/Blood-Bank-Management-System.
- Django framework documentation and project conventions.

5.3 Tools

- Microsoft Word for document preparation.
- draw.io / diagrams.net or equivalent for editable UML diagrams.
- Django development environment for implementation.
- Git and GitHub for source-code version control.

5.4 Division of Work

Team Member	SRN	Responsibility for SAD
KHUSH BHARDWAJ	PES2UG24CS226	Introduction, document overview, architecture goals/constraints, stakeholders, final review.
QAMAR AHMED	PES2UG24CS904	Component architecture, component descriptions, technology stack, architecture traceability.
KOUSHIK C	PES2UG24CS232	Architecture pattern, risks/mitigations, security architecture, API/interface design, error handling.
Hemaksh Breja	PES2UG24CS191	Sequence diagrams, UX/data design, open issues, references, final review.